Verilog 非阻塞赋值与时间槽调度

阻塞赋值、#0 延迟与 $display 执行顺序完整解析

一、题目

在一个 initial 块中，有如下几条语句，请分析仿真器打印出来的结果：

```
b = 0;
a <= b + 1;
#0;
$display(a, b);
```

二、背景知识：Time Slot 与 Region

Verilog/SystemVerilog 仿真器在每个时间步（Time Slot）内按固定顺序处理以下 Region：

Active Region	执行阻塞赋值（=）、NBA 的 RHS 采样、$display、连续赋值等。可循环多轮。
Inactive Region	#0 触发的事件在此等待，Active 清空后移回 Active 执行（回环）。
NBA Region	统一将所有非阻塞赋值（<=）的 LHS 写入，发生在 Active/Inactive 全部完成之后。
Postponed Region	$strobe 在此执行，保证读到本时间步所有赋值完成后的最终稳定值。

三、逐行执行分析

第1步 `b = 0` — Active 第1轮，阻塞赋值，b 立即变为 0。

第2步 `a <= b + 1` — Active 第1轮，RHS 采样得 1，LHS 放入 NBA 队列，a 仍为 x。

第3步 `#0` — 进程挂起，后续代码进入 Inactive Queue，等 Active 第1轮清空后回环到 Active 第2轮。

第4步 `$display(a, b)` — Active 第2轮（回环后）执行，NBA 尚未写入，a 仍是 x，b = 0。

第5步 `NBA Region 刷新` — a 被写入 1。但 $display 已执行完毕，看不到这个结果。

> 关键结论：$display 不是在 #0 之前执行的，而是经过 Inactive→Active 回环后才执行。NBA 的写入比回环更晚，因此两者都看不到 a=1。

四、时间槽执行示意图

下图展示 T=0ns（含 #0 回环）和 T=5ns（对比，无 NBA）两列的队列状态：



五、完整验证代码

将以下代码保存为 .v 文件，用 Icarus Verilog 运行：

```
iverilog -o out test.v && vvp out
```

T=0ns 验证：$display vs $strobe 对比

```
`timescale 1ns/1ps
module test;
  reg [7:0] a, b; // 使用8位，避免1位截断的bug

  initial begin
    b = 0; // Active 第1轮：阻塞赋值，b立即=0
    a <= b + 1; // Active 第1轮：RHS采样=1，入NBA队列
    #0; // 进程挂起，后续代码进入Inactive
                  // Inactive -> 回到Active第2轮
    $display("$display: a=%0d, b=%0d", a, b); // Active第2轮：a=x
    $strobe ("$strobe : a=%0d, b=%0d", a, b); // Postponed: a=1
  end

```

```
  initial #10 $finish;
endmodule
```

T=5ns 对比：无NBA，a 保持上轮写入的值

```
`timescale 1ns/1ps
module test;
  reg [7:0] a, b;

  // T=0ns: 有NBA，a最终写入1
  initial begin
    b = 0;
    a <= b + 1; // NBA队列: a <- 1
    #0;
    $display("T=0 $display: a=%0d, b=%0d", a, b); // a=x, b=0
  end

  // T=5ns: 无NBA，直接读上一轮写好的a
  initial begin
    #5;
    b = 5; // 阻塞赋值，b立即=5
    $display("T=5 $display: a=%0d, b=%0d", a, b); // a=1, b=5
  end

  initial #10 $finish;
endmodule
```

预期输出：

```
$display: a=x, b=0 // T=0，Active第2轮，NBA未写入
$strobe : a=1, b=0 // T=0，Postponed，NBA写完
T=0 $display: a=x, b=0
T=5 $display: a=1, b=5
```

六、核心规律总结

执行顺序	阻塞赋值（=）→ #0进Inactive → 回到Active → $display → NBA写入 → $strobe
#0 的作用	让进程挂起，后续代码进入 Inactive Queue，经回环后在下一轮 Active 执行
$display	跑在 Active Region，永远看不到同 time slot 的 NBA 写入结果
$strobe	跑在 Postponed Region，保证读到 NBA 写完后的最终稳定值
NBA 覆盖	若同一变量同一 time slot 既有 = 又有 <=，NBA 最终覆盖阻塞赋值的结果
1-bit 陷阱	reg a,b 是1位宽，b=5 会被截断为1，应使用 reg [7:0] 等多位宽声明

七、延伸问题：把 #0 换成 #1 会怎样？

将原题中的 #0 改为 #1，分析执行结果的变化。

题目代码：

```
`timescale 1ns/1ps
module test;
  reg [7:0] a, b;

  initial begin
    b = 0; // Active: 阻塞赋值，b立即=0
    a <= b + 1; // Active: RHS采样=1，入NBA队列
    #1; // 进程推进到T=1ns（新time slot）
    $display("a=%0d, b=%0d", a, b); // T=1ns: a=1, b=0
  end

  initial #10 $finish;
endmodule
```

实测输出：

```
a=1, b=0
```

与 #0 的核心区别：

#0	same time slot	进程进入同一 time slot 的 Inactive queue，回环到 Active 第2轮执行 $display。此时 NBA 尚未写入，a=x。
#1	next time slot	进程推入下一个 time slot（T=1ns）的 Future Events。T=0 的 Active 清空后 NBA 立即写入 a=1，$display 在 T=1ns 才运行，读到的已是稳定值 a=1。

时间槽执行示意图：



结论：

$display 能否看到 NBA 写入结果，取决于 $display 运行时所在的 time slot。#0 让进程留在同一 time slot，NBA 还没写；#1 让进程跳到下一个 time slot，NBA 早已写完。

八、常见混淆变体与 NBA 采样区域辨析

8.0 NBA 的 RHS 采样发生在哪个 Region？

一个容易混淆的问题：非阻塞赋值（<=）的 RHS 采样，究竟发生在 Active region 还是 Preponed region？

NBA RHS 采样	Active region	普通 RTL 代码中 <= 的右侧求值，发生在 Active region，和阻塞赋值、$display 在同一阶段。
Preponed region 采样	Preponed region	专门用于 SVA 并发断言（assert property）中的信号采样，每个 time slot 只执行一次，与普通 NBA 无关。

结论：普通 NBA 的 RHS 采样在 Active region。Preponed region 只服务于 SVA 并发断言，两者是完全不同的机制，不要混淆。

8.1 变体1：同一 always 块混用 = 和 <=

下面的代码能实现 a 和 b 的交换吗？

```
always @(posedge clk) begin
    a = b; // 阻塞：a 立即变为 b 的旧值
    b <= a; // RHS 采样此刻的 a，即 b 的旧值
end
```

结果：无法实现交换。每个时钟沿后，a 和 b 都等于 b 的旧值。

原因：b <= a 的 RHS 采样到的是阻塞赋值之后的 a，也就是 b 的旧值。两个变量最终值相同。若要正确交换，应全部改用 <=：

a <= b; b <= a; // 两个 RHS 都读各自旧值，交换正确实现。

8.2 变体2：多个 NBA 写同一变量

同一 initial 块里对同一变量连续两次 <=，最终结果是什么？

```
initial begin
    a <= 1; // 入NBA队列
    a <= 2; // 再次入NBA队列，覆盖上一条
end
```

结果：a = 2。

原因：IEEE 标准明确规定，同一 time slot 对同一变量的多次 NBA，最后一次入队的赋值胜出，前面的被覆盖。

8.3 变体3：NBA 的 RHS 依赖另一个 NBA 的结果

b 的 RHS 能读到 a <= 1 写入之后的值吗？

```
initial begin
    a <= 1; // RHS采样：1，入NBA队列
    b <= a + 1; // RHS采样：a 此刻仍是 x，b <- x+1
```

```
end
```

结果：b = x（不确定值）。

原因：两条语句的 RHS 都在 Active region 采样，此时 a 的 NBA 还没写入，a 仍是旧值 x。b 的 RHS 读到的是 x，而不是 1。这是 NBA 机制的核心特性：所有 RHS 采样发生在 Active region，LHS 统一在 NBA region 写入。

8.4 变体4：#0 之后再跟一个 NBA

#0 回环后又写了一次 a <= 2，最终 a 是 1 还是 2？

```
initial begin
    a <= 1; // Active第1轮：入NBA队列 a<-1
    #0; // 进入Inactive，回到Active第2轮
    a <= 2; // Active第2轮：再次入NBA队列 a<-2，覆盖
end
```

结果：a = 2。

原因：#0 回到 Active 第2轮后，a <= 2 再次入 NBA 队列，覆盖了第1轮的 a <= 1。等所有 Active/Inactive 轮次全部结束，NBA region 统一写入，最终 a = 2。

8.5 变体5：两个 initial 块同时对同一变量做 NBA

两个 initial 块同时执行，a 最终是 1 还是 2？

```
initial a <= 1;
initial a <= 2;
```

结果：不确定，是 race condition。

原因：两个 initial 块在同一 Active region 里，执行顺序由仿真器决定，标准不保证顺序。哪个 NBA 后入队哪个就赢，不同仿真器可能得到不同结果。这是典型的多进程写同一变量的竞争冒险，应避免此类写法。